
Développement d'applications distribuées

CER | Sécurité

Vendredi 12 juin 2026

TABLE DES MATIÈRES

AAV	3
Étude de cas : API	4
Mots clés	4
Contexte	4
Problématique	5
Contraintes	5
Généralisation	5
Livrables	5
Pistes de solutions	6
Plan d'actions	6
Définition : Mots clés	7
Migration en MSA	7
Full Stack	7
Application monolithique	8
Architecture en micro-services	8
API REST	9
Pile MERN	9
EKS	10
AWS	10
Orchestrateur	10
ODM (Mongoose)	11
Nginx	11
MongoDB	12
CRUD	12
API Gateway	13
Docker (container)	13
Scalability	14
CI/CD	14
Maturité de Richardson	15
Analyse et résolution : API	16
1. Du monolithe aux micro-services	16
2. Conception de l'API REST	17
3. Persistance et ORM JavaScript	17
4. Conteneurisation avec Docker	18
5. Propriétés non fonctionnelles	19
6. Déploiement et compatibilité cloud	19
7. Architecture retenue	20
Conclusion	21
Capitalisation	22

- 1 Citer les bonnes pratiques de déploiement d'API REST en utilisant Node.js
- 1 Citer les différents types d'infrastructures de déploiement
- 3 Définir les propriétés non fonctionnelles (fiabilité, performance, disponibilité, scalabilité, connectivité, évolutivité)
- 6 Expliquer l'utilisation d'un ORM Java-Script
- 3 Expliquer les possibilités d'architecture logicielle adaptée pour le projet (monolithiques -> Microservices)
- 3 Représenter l'architecture logicielle retenue de manière argumentée

ÉTUDE DE CAS : API

MOTS CLÉS

- Migration en MSA
- Full Stack
- Application monolithique
- Architecture en micro-services
- API REST
- pile MERN
- EKS
- AWS
- Orchestrateur
- ODM (Mongoose)
- Nginx
- MongoDB
- CRUD
- API Gateway
- Docker (container)
- Scalability
- CI/CD
- Maturité de Richardson

CONTEXTE

L'entreprise souhaite migrer progressivement son application monolithique vers une architecture en microservices. Le réseau social intégré à l'application de lecture et vente de bandes dessinées est utilisé comme projet pilote afin de développer un service backend exposant une API REST avec Node.js, Express et MongoDB.

PROBLÉMATIQUE

Comment concevoir une API REST, scalable et intégrable dans une architecture microservices tout en garantissant la séparation des services, la conteneurisation et la compatibilité avec une future infrastructure cloud ?

CONTRAINTES

- Node.js
- Express
- MongoDB
- Mongoose
- Niveau de maturité 2 (API)
- Docker
- Scalabilité
- API Gateway
- Load balancer Nginx

GÉNÉRALISATION

- API REST
- Microservices
- Conteneurisation
- Scalabilité
- Déploiement continu
- Persistance NoSQL

LIVRABLES

- Prototype CRUD gestion des posts
- Architecture logicielle documentée
- Exemples de bonnes pratiques de déploiement Docker

PISTES DE SOLUTIONS

- Architecture microservices avec Node.js et Express
- Les API REST pour la communication entre services
- Utilisation de MongoDB pour la persistance des données
- Conteneurisation avec Docker pour faciliter le déploiement
- Modèle Client-Serveur pour structurer les interactions

PLAN D'ACTIONS

- Définir les mots clés
- Etudier les principes de l'architecture microservices et des API REST
- Concevoir une architecture logicielle adaptée au projet
- Implémenter un prototype CRUD pour la gestion des posts
- Documenter l'architecture logicielle retenue
- Présenter des exemples de bonnes pratiques de déploiement avec Docker

DÉFINITION : MOTS CLÉS

Migration en MSA

La *Migration en MSA* (*MicroServices Architecture*) désigne le processus consistant à faire évoluer une application existante, généralement **monolithique**, vers une **architecture en micro-services**. Plutôt que de tout réécrire d'un seul bloc (approche risquée dite « Big Bang »), la migration est presque toujours **progressive** : on extrait un à un les modules métiers du monolithe pour en faire des services indépendants. Le *pattern* de référence est le « *Strangler Fig* » (figure de l'étrangleur) : les nouvelles fonctionnalités sont développées en micro-services tandis que l'ancien code est peu à peu « étranglé », jusqu'à disparaître. Dans le projet, le réseau social de l'application BD sert de **projet pilote** : c'est le premier domaine métier extrait du monolithe pour valider la démarche avant de l'étendre au reste de l'application.

Full Stack

Le terme *Full Stack* qualifie un développement (ou un développeur) couvrant l'**ensemble de la pile technique** d'une application, du **front-end** au **back-end** en passant par la base de données et le déploiement. On distingue trois couches :

- **Front-end** (client) : la partie visible, exécutée dans le navigateur (HTML, CSS, JavaScript, ici React).
- **Back-end** (serveur) : la logique métier et l'API (ici Node.js + Express).
- **Base de données** : la persistance des données (ici MongoDB).

La **pile MERN** (voir plus bas) est précisément une stack *Full Stack* où un seul langage, le JavaScript, est utilisé sur les trois couches, ce qui simplifie le développement et le partage de compétences.

Application monolithique

Une *application monolithique* est une application construite comme un **bloc unique et indivisible** : l'interface, la logique métier et l'accès aux données sont regroupés dans une seule base de code, compilée et déployée d'un seul tenant. Tous les modules partagent le même processus et, souvent, la même base de données.

Avantages : simplicité initiale, déploiement unique, communication interne rapide (appels de fonctions en mémoire). **Inconvénients** : couplage fort entre modules, montée en charge difficile (on doit dupliquer toute l'application même si un seul module est sollicité), une panne peut faire tomber l'ensemble, et tout changement impose de re-déployer la totalité — ce qui freine l'**évolutivité** et l'agilité des équipes. C'est précisément ce qui motive la migration vers les micro-services.

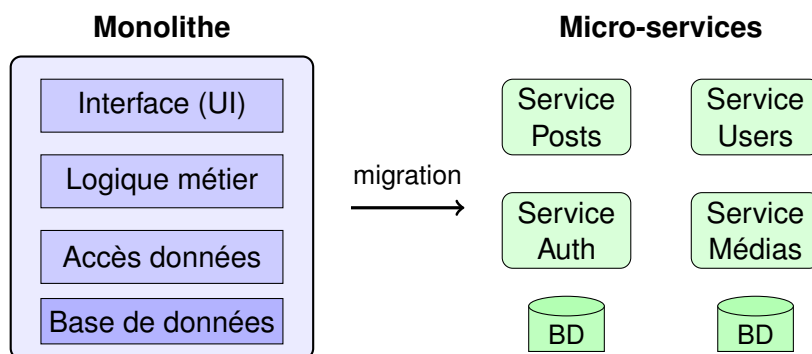


Figure : du bloc monolithique unique vers des services indépendants disposant chacun de sa propre base de données.

Architecture en micro-services

L'*architecture en micro-services* (MSA) consiste à découper une application en un ensemble de **petits services autonomes**, chacun responsable d'une seule capacité métier (gestion des posts, des utilisateurs, de l'authentification, etc.). Chaque service :

- est **développé, déployé et mis à l'échelle indépendamment** des autres ;
- possède idéalement sa **propre base de données** (*Database per Service*) ;
- communique avec les autres via des **interfaces réseau légères**, le plus souvent des **API REST** en HTTP/JSON.

Avantages : **scalabilité** fine (on ne met à l'échelle que le service sollicité), **résilience** (une panne reste isolée), liberté technologique par service, déploiements indépendants et plus fréquents (**évolutivité**). **Inconvénients** : complexité accrue (réseau, supervision, cohérence des données distribuées), nécessité d'outils d'orchestration et d'une **API Gateway**.

API REST

Une **API** (*Application Programming Interface*) est un contrat qui permet à deux logiciels de dialoguer. **REST** (*REpresentational State Transfer*) est un **style d'architecture** pour concevoir des API au-dessus du protocole **HTTP**. Une API REST manipule des **ressources** (ex. : un *post*, un *user*) identifiées par des **URI** (ex. : `/api/posts/42`), sur lesquelles on applique les **verbes HTTP** :

Verbe HTTP	Action CRUD	Exemple
GET	Lire (Read)	GET <code>/api/posts</code>
POST	Créer (Create)	POST <code>/api/posts</code>
PUT / PATCH	Mettre à jour (Update)	PUT <code>/api/posts/42</code>
DELETE	Supprimer (Delete)	DELETE <code>/api/posts/42</code>

Les principes clés de REST sont : l'architecture **Client-Serveur**, le caractère **sans état** (*stateless* : chaque requête contient toute l'information nécessaire, le serveur ne mémorise pas la session), une **interface uniforme**, et l'échange de représentations standardisées (généralement du **JSON**).

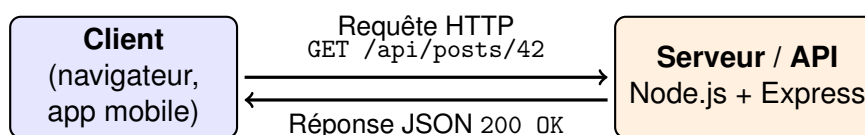


Figure : échange Client-Serveur sans état au cœur d'une API REST.

Pile MERN

La **pile MERN** est un ensemble de quatre technologies **tout-JavaScript** permettant de construire une application web *Full Stack* :

- **M – MongoDB** : base de données NoSQL orientée documents (persistance).
- **E – Express** : framework web minimaliste pour Node.js (routes et API REST).
- **R – React** : bibliothèque front-end pour l'interface utilisateur (côté client).
- **N – Node.js** : environnement d'exécution JavaScript côté serveur.

Son intérêt majeur : un **langage unique (JavaScript)** de bout en bout, et un format de données homogène (**JSON / BSON**) du navigateur jusqu'à la base, ce qui réduit les frictions et facilite le travail des équipes *Full Stack*.

EKS

EKS (*Elastic Kubernetes Service*) est le service **managé Kubernetes** proposé par **AWS**. Il fournit un cluster **Kubernetes** (l'orchestrateur de référence) dont AWS gère pour nous le *control plane* (le « cerveau » qui pilote les conteneurs), la haute disponibilité et les mises à jour. EKS permet de **déployer, orchestrer et mettre à l'échelle** automatiquement des conteneurs Docker dans le **cloud**. Dans le contexte du projet, EKS représente l'**infrastructure cible** permettant d'héberger les micro-services conteneurisés de façon scalable et résiliente.

AWS

AWS (*Amazon Web Services*) est la plateforme de **cloud computing** d'Amazon, leader du marché. Elle propose, à la demande et facturé à l'usage (*pay-as-you-go*), une vaste gamme de services : **calcul** (EC2), **conteneurs/orchestration** (ECS, **EKS**), **stockage** (S3), **bases de données**, **réseau** et **équilibrage de charge**. AWS illustre l'**infrastructure de type IaaS/PaaS** (« cloud public ») vers laquelle le projet doit rester compatible, par opposition à un hébergement « on-premise » (sur les serveurs de l'entreprise).

Orchestrateur

Un **orchestrateur** de conteneurs est un outil qui **automatise le déploiement, la gestion, la mise à l'échelle et le réseau** d'un grand nombre de conteneurs. Là où Docker lance un conteneur, l'orchestrateur gère une **flotte** de conteneurs répartis sur plusieurs machines. Le standard du marché est **Kubernetes** (souvent abrégé **K8s**). Ses fonctions principales :

- **Scaling automatique** : ajout/retrait de conteneurs selon la charge.
- **Auto-réparation** : redémarre un conteneur défaillant (**fiabilité / disponibilité**).
- **Load balancing** : répartit le trafic entre les instances.
- **Déploiements progressifs** (*rolling updates*) sans interruption de service.

C'est l'élément indispensable pour exploiter une architecture micro-services à grande échelle (cf. **EKS**).

ODM (Mongoose)

Un **ODM** (*Object-Document Mapping*) est l'équivalent d'un **ORM** (*Object-Relational Mapping*) mais pour les bases **NoSQL orientées documents** comme MongoDB. Il fait le pont entre le **monde objet** du code (objets JavaScript) et les **documents** stockés en base, évitant d'écrire des requêtes bas niveau.

Mongoose est l'ODM de référence pour MongoDB en Node.js. Il apporte :

- des **Schémas** qui définissent la structure et les types des documents ;
- une **validation** automatique des données avant insertion ;
- des **modèles** offrant des méthodes CRUD prêtes à l'emploi (*find, create, ...*) ;
- la gestion des relations, des *hooks* et des requêtes typées.

C'est l'illustration directe de l'AAV « Expliquer l'utilisation d'un ORM JavaScript » : Mongoose abstrait l'accès aux données, fiabilise le code et accélère le développement.

Nginx

Nginx (prononcé « engine-x ») est un **serveur web** hautes performances qui joue, dans une architecture distribuée, deux rôles essentiels :

- **Reverse proxy** : il se place devant les serveurs applicatifs et reçoit les requêtes des clients pour les transmettre aux bons services (point d'entrée unique, terminaison TLS/HTTPS).
- **Load balancer** (équilibreur de charge) : il **répartit le trafic** entre plusieurs instances d'un même service, ce qui améliore la **performance**, la **disponibilité** et permet la **scalabilité horizontale**.

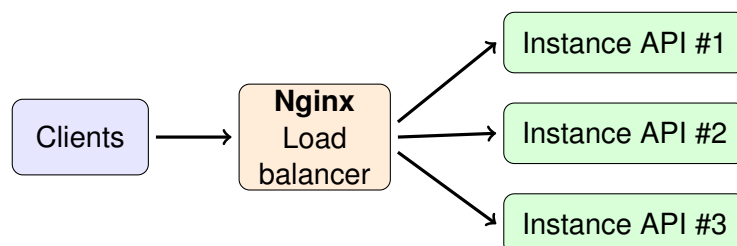


Figure : Nginx répartit la charge entre plusieurs instances de l'API (scalabilité horizontale).

MongoDB

MongoDB est un **système de gestion de base de données NoSQL** orienté **documents**. Au lieu de tables et de lignes (modèle relationnel), il stocke les données sous forme de **documents JSON/BSON** flexibles, regroupés en **collections**. Exemple d'un document *post* :

```
{ "_id": 42, "auteur": "Alice", "contenu": "...", "likes": 12 }
```

Atouts : **schéma flexible** (idéal pour des données évolutives), excellente **scalabilité horizontale** (*sharding*), bonnes performances en lecture/écriture, et homogénéité avec JavaScript (format JSON). C'est le « M » de la pile **MERN** et la solution de **persistance NoSQL** retenue, avec un principe *Database per Service* dans l'architecture micro-services.

CRUD

CRUD est l'acronyme des **quatre opérations fondamentales** de manipulation de données persistées :

- **C – Create** (créer) — POST
- **R – Read** (lire) — GET
- **U – Update** (mettre à jour) — PUT/PATCH
- **D – Delete** (supprimer) — DELETE

Ces opérations correspondent directement aux **verbes HTTP** d'une API REST et aux méthodes d'un ODM comme Mongoose. Le **livrable du projet** est précisément un **prototype CRUD de gestion des posts** du réseau social.

API Gateway

Une **API Gateway** (passerelle d'API) est le **point d'entrée unique** d'une architecture micro-services. Tous les appels clients passent par elle, et elle se charge de les **router** vers le bon micro-service. Elle centralise des préoccupations transverses qu'il serait coûteux de dupliquer dans chaque service :

- **routing** et agrégation des requêtes ;
- **authentification / autorisation** (ex. : vérification des jetons JWT) ;
- **limitation de débit** (*rate limiting*), journalisation, mise en cache ;
- **terminaison TLS** et masquage de la structure interne (**connectivité** simplifiée pour le client).

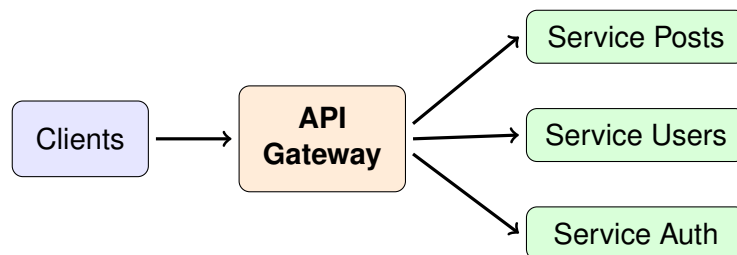


Figure : l'API Gateway, point d'entrée unique routant les requêtes vers les micro-services.

Docker (container)

Docker est une plateforme de **conteneurisation**. Un **conteneur** est un paquet **léger et isolé** qui embarque une application **avec toutes ses dépendances** (bibliothèques, runtime, configuration), garantissant qu'elle s'exécute **de façon identique** partout (« ça marche sur ma machine » devient « ça marche partout »).

À la différence d'une **machine virtuelle (VM)** qui embarque un système d'exploitation complet, les conteneurs **partagent le noyau de l'OS hôte** : ils sont donc beaucoup plus **légers, rapides à démarrer et économes** en ressources.

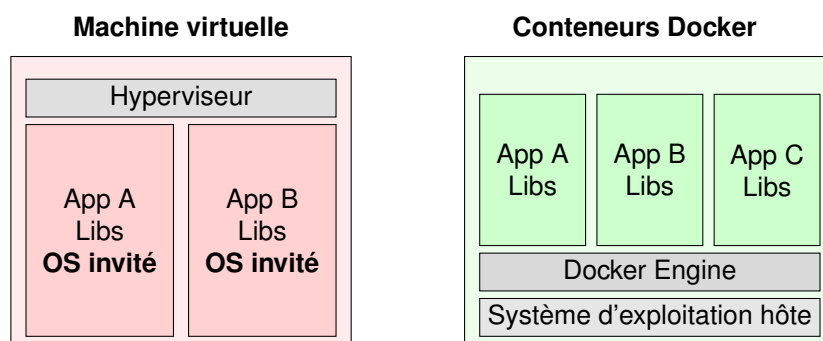


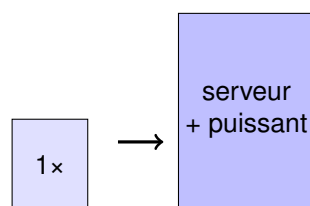
Figure : la VM virtualise tout un OS, le conteneur partage le noyau hôte (plus léger).

Scalability

La **scalabilité** (ou *passage à l'échelle*) est la capacité d'un système à **absorber une augmentation de charge** (plus d'utilisateurs, plus de requêtes) en maintenant ses performances. On distingue deux stratégies :

- **Scalabilité verticale** (*scale up*) : ajouter de la puissance à une même machine (plus de CPU, RAM). Simple, mais limitée et coûteuse.
- **Scalabilité horizontale** (*scale out*) : **multiplier les instances** d'un service et répartir la charge entre elles (via un load balancer). C'est l'approche privilégiée en micro-services car elle est quasi illimitée et résiliente.

Verticale (scale up)



Horizontale (scale out)

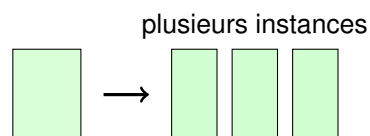


Figure : scalabilité verticale (machine plus puissante) vs horizontale (plus d'instances).

CI/CD

CI/CD désigne une chaîne d'**automatisation** du cycle de livraison logicielle :

- **CI – Continuous Integration** (intégration continue) : à chaque *commit*, le code est automatiquement **compilé et testé** (tests unitaires, analyse qualité), ce qui détecte les régressions au plus tôt.
- **CD – Continuous Delivery / Deployment** (livraison/déploiement continu) : les versions validées sont automatiquement **packagées** (image Docker) et **déployées** sur les environnements (test, production).

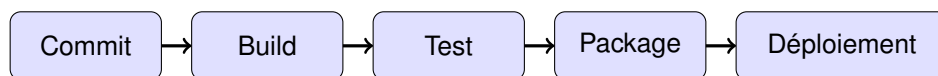


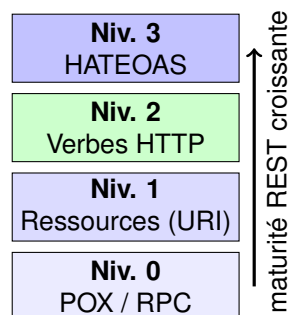
Figure : pipeline CI/CD — du commit au déploiement automatisé.

Outils courants : **GitHub Actions**, **GitLab CI**, **Jenkins**, **Azure DevOps**. Le CI/CD est au cœur des **bonnes pratiques de déploiement** et du **déploiement continu** visés par le projet.

Maturité de Richardson

Le **modèle de maturité de Richardson** (*Richardson Maturity Model*, RMM) est une échelle, proposée par Leonard Richardson, qui mesure à quel point une API respecte réellement les principes **REST**. Elle comporte **4 niveaux** :

- **Niveau 0 – « Le marais de POX »** : une seule URL, un seul verbe (souvent POST), HTTP est juste un tunnel de transport (style RPC).
- **Niveau 1 – Ressources** : introduction de multiples **URI** identifiant chaque ressource (/posts, /users), mais sans usage correct des verbes.
- **Niveau 2 – Verbes HTTP** : utilisation correcte des **verbes** (GET, POST, PUT, DELETE) et des **codes de statut** (200, 404, 201...). **C'est le niveau visé par le projet** (contrainte « niveau de maturité 2 ») et celui de la grande majorité des API REST en production.
- **Niveau 3 – HATEOAS** : les réponses contiennent des **hyperliens** décrivant les actions possibles suivantes ; l'API devient auto-découvrable. C'est le REST « pur ».



« Niveau 2 » = cible du projet :
verbes + codes HTTP corrects,
standard de fait des API REST.

ANALYSE ET RÉOLUTION : API

Rappel de la problématique : « Comment concevoir une API REST, scalable et intégrable dans une architecture micro-services tout en garantissant la séparation des services, la conteneurisation et la compatibilité avec une future infrastructure cloud ? »

Pour répondre, nous traitons successivement les cinq facettes de la problématique : **(1)** le choix de l'architecture logicielle, **(2)** la conception de l'API REST et la séparation des services, **(3)** la persistance via un ORM/ODM JavaScript, **(4)** la conteneurisation, et **(5)** le déploiement et la compatibilité cloud. Une synthèse argumentée de l'architecture retenue clôt l'analyse.

1. DU MONOLITHE AUX MICRO-SERVICES : L'ARCHITECTURE RETENUE

Deux grandes familles d'architecture s'offraient au projet :

- **Monolithique** : simple à démarrer, mais couplage fort, mise à l'échelle globale (on duplique *tout* même pour un seul module surchargé) et re-déploiement total à chaque changement — un frein direct à la **scalabilité** et à l'**évolutivité** exigées.
- **Micro-services (MSA)** : chaque capacité métier devient un service autonome, **déployé et mis à l'échelle indépendamment**, avec sa **propre base de données** (*Database per Service*).

Décision : l'architecture **micro-services** est retenue car elle satisfait directement les contraintes (scalabilité fine, résilience, déploiements indépendants). La migration n'est pas un « Big Bang » mais **progressive** (*pattern Strangler Fig*) : le **réseau social** de l'application BD est extrait en premier comme **projet pilote**, validant la démarche avant d'étendre le découpage au reste du monolithe.

2. CONCEPTION DE L'API REST ET SÉPARATION DES SERVICES

Le service pilote expose une **API REST** de **niveau 2 du modèle de maturité de Richardson** (contrainte projet) : des **ressources** identifiées par des URI, manipulées par les **verbes HTTP** corrects et renvoyant les bons **codes de statut**. Pour la ressource `post` :

Verbe + URI	Action	Code attendu
GET /api/posts	Lister les posts	200 OK
GET /api/posts/:id	Lire un post	200 / 404
POST /api/posts	Créer un post	201 Created
PUT /api/posts/:id	Mettre à jour un post	200 / 404
DELETE /api/posts/:id	Supprimer un post	204 No Content

La **séparation des services** est garantie par : le caractère **sans état** (*stateless*) de REST — chaque requête est autosuffisante, aucun service ne partage de session — et le principe *Database per Service* qui interdit l'accès direct à la base d'un autre service. Toute communication inter-services passe par des **API REST en HTTP/JSON**, ce qui maintient un **couplage faible**.

3. PERSISTANCE NOSQL ET ORM JAVASCRIPT (MONGOOSE)

La persistance repose sur **MongoDB** (NoSQL orienté documents) avec, comme **ORM/ODM JavaScript**, **Mongoose**. Un ODM fait le pont entre les **objets JavaScript** du code et les **documents** stockés, en évitant d'écrire des requêtes bas niveau. Concrètement, on définit un **schéma** (structure + validation), puis un **modèle** qui fournit des méthodes CRUD prêtes à l'emploi :

```
const mongoose = require('mongoose');

// 1) Schéma : structure et règles de validation du document
const postSchema = new mongoose.Schema({
  auteur: { type: String, required: true },
  contenu: { type: String, required: true, maxlength: 500 },
  likes: { type: Number, default: 0 },
}, { timestamps: true }); // ajoute createdAt / updatedAt

// 2) Modèle : objet manipulable offrant les opérations CRUD
const Post = mongoose.model('Post', postSchema);
```

```
// 3) Exemple d'usage CRUD dans un contrôleur Express
async function creerPost(req, res) {
  try {
    const post = await Post.create(req.body); // CREATE -> valide puis insère
    res.status(201).json(post); // 201 Created
  } catch (err) {
    res.status(400).json({ erreur: err.message }); // validation échouée
  }
}

async function listerPosts(req, res) {
  const posts = await Post.find(); // READ
  res.status(200).json(posts);
}
```

Apports de l'ODM dans le projet : validation automatique avant insertion (fiabilité des données), **abstraction** de l'accès aux données (code plus lisible et maintenable), méthodes CRUD (create, find, updateOne, deleteOne) qui correspondent directement aux **verbes HTTP**, et homogénéité **tout-JavaScript / JSON** du client jusqu'à la base (pile MERN). C'est la réponse concrète à l'AAV « Expliquer l'utilisation d'un ORM JavaScript ».

4. CONTENEURISATION AVEC DOCKER

Chaque micro-service est **conteneurisé** avec Docker : il embarque l'application **et toutes ses dépendances**, garantissant une exécution **identique** du poste de développement à la production. Un Dockerfile décrit l'image du service :

```
FROM node:20-alpine # image légère
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev # dépendances de production uniquement
COPY . .
EXPOSE 3000
CMD ["node", "server.js"] # point d'entrée du service
```

Bonnes pratiques appliquées : image de base **Alpine** (légereté), copie du package.json **avant** le code pour profiter du cache de couches, npm ci (build reproductible), et séparation d'un conteneur par service. En développement, **Docker Compose** orchestre localement le service + une instance MongoDB.

5. PROPRIÉTÉS NON FONCTIONNELLES

L'architecture retenue est évaluée au regard des **propriétés non fonctionnelles** attendues :

Propriété	Comment elle est satisfaite
Fiabilité	Validation Mongoose des données ; auto-réparation des conteneurs par l'orchestrateur.
Performance	Node.js non-bloquant ; Nginx en cache/reverse proxy ; MongoDB performant en lecture/écriture.
Disponibilité	Plusieurs instances par service ; redémarrage automatique ; load balancing.
Scalabilité	Scalabilité horizontale (scale out) service par service via l'orchestrateur.
Connectivité	Point d'entrée unique (API Gateway / Nginx) ; échanges normalisés HTTP/JSON.
Évolutivité	Services déployables indépendamment ; ajout d'un nouveau service sans toucher aux autres.

6. INFRASTRUCTURES DE DÉPLOIEMENT ET COMPATIBILITÉ CLOUD

On distingue plusieurs **types d'infrastructures de déploiement** :

- **On-premise** (serveurs de l'entreprise) : contrôle total, mais élasticité limitée.
- **IaaS** (machines virtuelles dans le cloud, ex. AWS EC2) : souplesse, on gère l'OS.
- **PaaS** (plateforme managée) : on ne gère que l'application.
- **Conteneurs orchestrés** (Kubernetes / **AWS EKS**) : déploiement, scaling et résilience automatisés — **cible du projet**.
- **Serverless** (FaaS) : exécution à la demande, sans serveur à gérer.

Bonnes pratiques de déploiement d'API REST Node.js : variables d'environnement pour la configuration (pas de secret en dur), pipeline **CI/CD** (build → test → image Docker → déploiement), versionnement de l'API (`/api/v1`), journalisation et *health checks*, et déploiements progressifs (*rolling updates*) sans interruption. La conteneurisation rend l'application **portable** : la même image tourne en local (Docker Compose) puis sur **EKS**, assurant la **compatibilité avec la future infrastructure cloud** sans réécriture.

7. ARCHITECTURE LOGICIELLE RETENUE (SCHÉMA ARGUMENTÉ)

L'architecture cible combine tous les éléments précédents : les clients passent par une **API Gateway / Nginx** (point d'entrée unique, routage, load balancing), qui distribue les requêtes vers des **micro-services conteneurisés**, chacun disposant de sa **propre base MongoDB**, le tout orchestré sur **AWS EKS**.

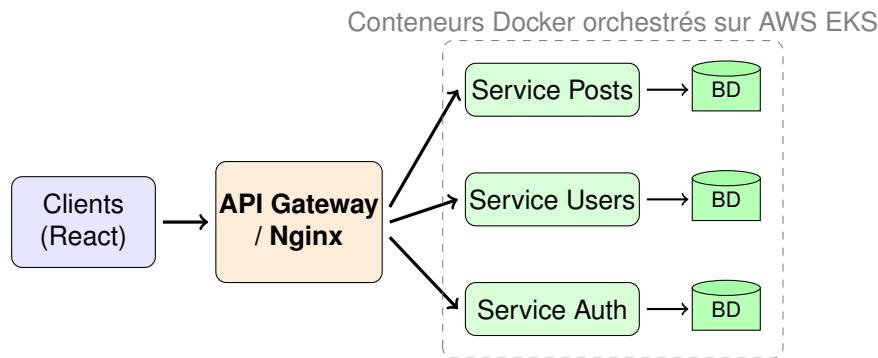


Figure : architecture retenue — API Gateway/Nginx en façade, micro-services conteneurisés avec base par service, orchestrés sur EKS.

Justification : cette architecture répond à chaque exigence de la problématique — REST de niveau 2 (intégrabilité), *Database per Service* (séparation), Docker (conteneurisation), scalabilité horizontale par l'orchestrateur, et portabilité vers le cloud via EKS.

CONCLUSION

En réponse à la problématique, concevoir une API REST scalable et intégrable dans une architecture micro-services passe par un ensemble de choix cohérents et complémentaires. Le service pilote — le réseau social de l'application BD — est extrait du monolithe selon une démarche progressive (*Strangler Fig*) et exposé sous forme d'une **API REST de niveau 2**, ce qui en fait une brique **intégrable** et faiblement couplée. La **séparation des services** est assurée par le caractère *stateless* de REST et le principe *Database per Service*, chaque micro-service possédant sa propre base **MongoDB** pilotée par l'ODM **Mongoose**, qui fiabilise et simplifie l'accès aux données.

La **conteneurisation** avec Docker garantit une exécution identique de l'environnement de développement jusqu'à la production, tandis que l'orchestration sur **AWS EKS** apporte la **scalabilité horizontale**, la **disponibilité** et la **résilience** attendues, le tout exposé derrière une **API Gateway / Nginx** faisant office de point d'entrée unique et d'équilibreur de charge. Parce que l'image conteneurisée est portable, la **compatibilité avec la future infrastructure cloud** est acquise sans réécriture.

Au final, l'architecture micro-services retenue satisfait l'ensemble des propriétés non fonctionnelles visées (fiabilité, performance, disponibilité, scalabilité, connectivité, évolutivité) et constitue un **modèle reproductible** : la démarche validée sur ce projet pilote pourra être étendue, service après service, au reste de l'application monolithique de l'entreprise.

CAPITALISATION

AAV	Commentaire	Statut
Citer les bonnes pratiques de déploiement d'API REST en utilisant Node.js	Bonnes pratiques détaillées : variables d'environnement, CI/CD, versionnement d'API, <i>health checks</i> , <i>rolling updates</i> , image Docker portable.	Acquis
Citer les différents types d'infrastructures de déploiement	On-premise, IaaS (EC2), PaaS, conteneurs orchestrés (Kubernetes/EKS) et serverless présentés et comparés.	Acquis
Définir les propriétés non fonctionnelles (fiabilité, performance, disponibilité, scalabilité, connectivité, évolutivité)	Les six propriétés sont définies et reliées, via un tableau, aux choix d'architecture qui les satisfont.	Acquis
Expliquer l'utilisation d'un ORM JavaScript	Rôle d'un ODM (Mongoose) expliqué : schéma, validation, modèle et CRUD, illustré par un exemple de code commenté.	Acquis
Expliquer les possibilités d'architecture logicielle adaptée pour le projet (monolithique → micro-services)	Comparaison monolithe / micro-services argumentée ; migration progressive (<i>Strangler Fig</i>) et projet pilote justifiés.	Acquis
Représenter l'architecture logicielle retenue de manière argumentée	Architecture cible schématisée (API Gateway/Nginx, micro-services conteneurisés, base par service, EKS) et justifiée point par point.	Acquis

RESSOURCES

- **Documentation Node.js** : <https://nodejs.org/docs/latest/api/>
- **Documentation Express** : <https://expressjs.com/fr/>
- **Documentation MongoDB** : <https://www.mongodb.com/docs/>
- **Documentation Mongoose (ODM)** : <https://mongoosejs.com/docs/>
- **REST / maturité de Richardson** : <https://martinfowler.com/articles/richardsonMaturityModel.html>
- **Documentation Docker** : <https://docs.docker.com/>
- **Amazon EKS (Kubernetes managé)** : <https://docs.aws.amazon.com/eks/>
- **Microservices (patterns)** : <https://microservices.io/patterns/>